

1. 定理いろいろ

このパズルを解く際に有用な定理を紹介します。

1. マッチ棒だけでなく、頂点に結びついているマッチ棒の数（これ以降は次数と表記）も頂点ごとにそろえる問題としてよい。つまり、次数が違ふ頂点はそれぞれ最後に置かれる場所が異なる。

例として、ある盤面において次数が2の頂点を考える。その頂点が最後に移動する可能性がある位置は、一致させようとしているパターンの頂点の位置の内、その位置の頂点の次数が2である位置のみである。つまり、次数が3や0である頂点の位置に移動することはない。

また、揃えたいパターンにある同じ次数の頂点の個数と操作するマッチ棒にある同じ次数の頂点の個数は必ず一致する。つまり、揃えたいパターンと操作するマッチ棒の頂点の次数の多重集合は必ず一致する。（そうでなければその盤面のパズルは解くことができない）

例として適当な盤面の次数のみを表記した表を考える。（今はマッチ棒のつながり方は考えない）

	列 1	列 2	列 3	列 1'	列 2'	列 3'
行 1	0	1	2	4	1	2
行 2	1	2	4	2	1	0
行 3	2	0	1	0	1	2

例えば1行2列目の次数が1の頂点を考える。この頂点はパターンの頂点の次数から1行2列目か2行2列目、3行2列目のどれかに移動することが分かる。

（雑談：実はそれぞれの頂点がこの次数になる盤面は存在しない。）

2. 次数が0の頂点は（上の条件をみたすなら、つまり、一致させようとしているパターンの次数が0の頂点の位置のどれかであるなら、）どのように並び替えてもパターンと一致する。

なぜなら、結びついているマッチ棒がないので次数が0の頂点同士を区別することができないからだ。例として適当な盤面の次数のみを表記した表を考える。（上の例と同じ表）

	列 1	列 2	列 3	列 1'	列 2'	列 3'
行 1	0	1	2	4	1	2
行 2	1	2	4	2	1	0
行 3	2	0	1	0	1	2

このとき1行1列目の次数が0の頂点は3行1列目、2行3列目のどちらに動かしてもよい。

ただし、最短手順になる保証はない。例として、頂点の次数が0,1,0,2,1で、一致させるパターンが1,0,2,0,1であるとき、

1,0,0,2,1 (1,2)

1,0,2,0,1 (3,4)

と入れ替えると二手で最短（一回の交換では二個までしかそろえられないから）だが、

2,1,0,0,1 (1,4)
2,0,1,0,1 (2,3)
1,0,2,0,1 (1,3)

と入れ替えてしまうと、二手で揃わず三手かかってしまう。あくまで、どの0を持って行っても、その0をこれ以上動かさずに解くことができるというだけである。

3. ある頂点を考えたとき、その頂点の次数と同じである頂点がほかに存在しないとき、その頂点の位置は一意に定まる。

なぜなら、次数が違う頂点はそれぞれ異なる場所に置かれるから、その頂点が移動する可能性がある位置は一つしかなく一意に定まるからだ。このとき、既に位置が確定した頂点は存在しないものとして考えられる。

例えば上の表では、1行3列目の次数が4の頂点は1行1列目に動かすしかない。

4. すでに位置が確定したある頂点を考える。この頂点から一本のマッチ棒でつながっている（以降、隣接すると表記）頂点の集合を考える。このときその集合に含まれるある頂点の次数が、その集合の内の自分自身以外のどの頂点の次数とも異なる場合、その頂点の位置は一意に定まる。
5. 同じ頂点に隣接している次数1の頂点の位置は互いに交換可能である。したがって、同じ頂点に隣接している次数1の頂点をまとめて考えることができる。

ただし、次数が0の時と同様に、最短手順であることは保証しない。スタートが0,1,2,1、ゴールが1,0,1,2であるとき、(1,2)の入れ替えで始めると二手で揃うが、(1,4)は二手で揃わず三手かかる。

6. 同じ次数を持った頂点をすべて集めた集合に対し、それぞれの頂点の次数の多重集合（その頂点に隣接する頂点の次数を集めたもの）を考える。

このとき、違う次数の多重集合を持った頂点はそれぞれ置ける位置が異なる。また、頂点の持っている次数の多重集合が他の頂点のどれとも異なる場合、その頂点の位置は一意に定まる。

7. 確定した各頂点にユニークなidをつける。このとき、確定した頂点のどれかに隣接する頂点の集合を考える。このとき、違うidの頂点に隣接する頂点はそれぞれ置ける位置が異なる。

また、確定していない頂点の中で、確定した頂点に隣接する頂点のidの集合が他の頂点のどれとも異なる場合、その頂点の位置は一意に定まる。

8. 入れ替えた後の次数がどちらもパターンの次数と一致しない頂点の入れ替えはしなくてよい（枝狩りしてよい）。ここから先の説明は節5を前提にしているので、先にそちらを読んでおくことを推奨する。

正しくない位置にいる頂点を正しい位置に一つずつ移動していく形の最短の解が存在する。これは、 C が一番大きくなるような順列の一つを選び、その順列に従って移動させればよい。

操作は全て、 k 番目にいない k を k 番目に入れ替える操作となるので、前述の説明と同様に C が一つ増える。操作前は $k \rightarrow k$ 番目にいる要素 $\rightarrow \dots \rightarrow k$ の一つ前 $\rightarrow k$ というサイクルがあり、操作後は $k \rightarrow k$ のサイクルと、 k 番目にもともといた要素 $\rightarrow \dots \rightarrow k$ の一つ前だった要素 $\rightarrow k$ 番目にもともといた要素の二つに分裂する。従って C が一つ増加する。

この定理で求めた解は、少なくとも、入れ替えた後の次数のどちらかがパターンの次数と一致する。したがって、入れ替えた後の次数がどちらもパターンの次数と一致しない頂点の入れ替えはしなくても、最短の解は求められることが保証される。

この定理は人力で解くときには使いにくい、プログラムで解くときには有用である。

2. 戦略例

// できる限り繰り返す。

while True:

 if すべての頂点の位置が確定している:

 // 終了

 return

 if 確定していない頂点の中で、次数がユニークな頂点がある:

 // その頂点の位置は一意に定まる。

 その頂点を確定させる。(定理2,4)

 continue

既に位置が確定している頂点を次数順にソート

for すでに位置が確定している頂点v in すべての頂点:

 if 隣接する位置が確定していない頂点の中で、次数がユニークな頂点がある:

 その頂点を確定させる。(定理5)

 break

if 上のfor文で頂点が確定した:

 continue

for 確定していない頂点v in すべての頂点:

 if 他の確定していないかつ同じ次数の頂点の中に、隣接する頂点の次数の多重集合が一致するものがない:
 その頂点を確定させる。(定理7)

 break

if 上のfor文で頂点が確定した:

 continue

for 確定していない頂点v in すべての頂点:

 if 他の確定していないかつ同じ次数の頂点の中に、隣接する確定した頂点のidの集合が一致するものがない:

 その頂点を確定させる。(定理8)

 break

if 上のfor文で頂点が確定した:

 continue

break // これ以上確定できる頂点がないので、while文を抜ける

// 最短の手順が保証されるのはここまで

次数0の頂点を移動させる。(定理3)

確定している頂点に隣接する次数1の頂点を移動させる。(定理6)

解が完成しているかを確認する。完成していなければ、その盤面はこのルールベースの戦略では解くことができない盤面である。

return

最短を目指さないのであれば、次数0の頂点を移動させることは一番最初に行うのがいいだろう。その方が頭に入れておくことが減って解きやすい。

3. 元ネタ

グラフ同型判定問題 ([Wikipedia\(日本語\)](#)、[Wikipedia\(English\)](#))

一般的なパズルは NP 完全なことが多いらしいが、この問題が NP 完全に属するかは未解決問題。ちなみに今回のパズルは同型であることが分かったうえで、その同型写像たちを最小の互換で作るような問題なので厳密には別物である。

4. 総盤面数

例えば、難易度が Normal だと、右のグラフで、辺が生成されうる場所が $9 \cdot \frac{8}{2} = 36$ なので 2^{36} 以上の盤面があり、右の盤面を固定した時の左の盤面数が重複を含め多くとも $9!$ なので $2^{36} \cdot 9!$ 以下になるところまで判明した。厳密な値を出すのは諦めた。

5. 最短手数

頂点数が少なければ、1 回の交換で到達できる盤面、2 回の交換で到達できる盤面、3 回の交換で到達できる盤面、...

というような形で順に BFS をしていき、それぞれの盤面が目標のグラフと同じになっているかを判定すれば求められる。

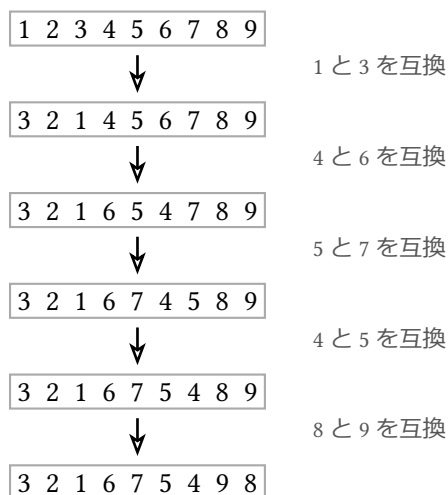
頂点数が多い場合は、状態数が多くなりすぎるため、BFS をしてもメモリが足りなくなってしまうため、ヒューリスティック的な方法で最短手数を求める必要がある。[strategy.md](#) を参照。

また、最短手数は頂点数以下である。例えば、Normal での最短手数は 9 手以下である。

これは、初期状態の 9 頂点から、全ての状態への移行がなんらかの置換で表され、これらの任意の置換が 9 つの互換で合成できることから従う。これを証明する。

頂点に 1、2、...、9 とインデックスをふる。このとき、頂点の任意の置換は 1 から 9 の順列で表される。この置換を先頭から作ることを考える。

例として解の一つが 3、2、1、6、7、5、4、9、8 であったとき、この置換を先頭から作る。



この例では、先頭から順に位置を確定させ、使用した互換は 5 個である。

このように先頭から一つずつそろえることで、一つ揃えるたびに最大でも一つしか互換が必要ないことから、最短手数が 9 手以下になることがいえる。(ちなみに、最後の一回分は自動的にそろうので最短手数は 8 手以下も証明できる。)

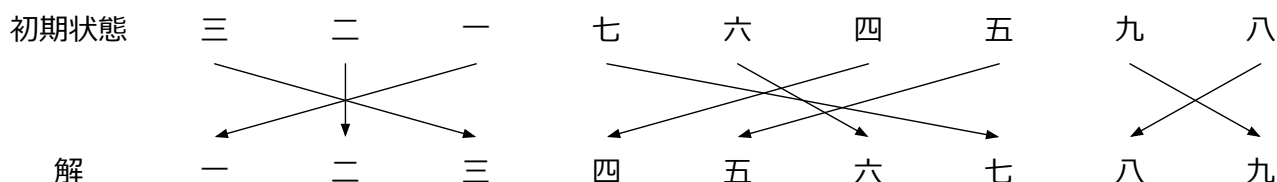
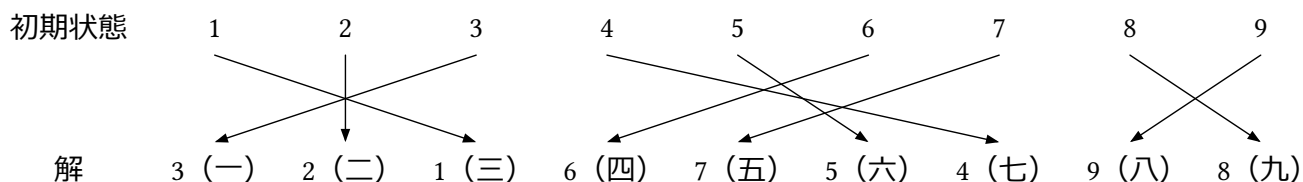
さらに、行うことが確定している操作（ある頂点を置く場所が一か所に決まっている状態のこと、詳しくは [strategy.md](#) を参照）を繰り返すだけで多くの場合最短手数を達成できる。

(少なくとも、行うことが確定している操作がなくなるまではその操作を続けることが最短手順の一つ)

5.1. 説明

5.1.1. 問題の簡略化

ここからは問題の番号をつけなおして、列を昇順に整列する問題になるように考える。例えば、1、2、3、4、5、6、7、8、9から3、2、1、6、7、5、4、9、8を作る場合は、次のように考えられる。



1 が三番目に来ているから、1 の場所を三にすると、昇順にソートしたときに三が三番目に来るようになる。同様に繰り返すと、最終的には、三、二、一、七、六、四、五、九、八に並んだ列を昇順にソートする問題になる。

一般的には、解の i 番目にいる数字が i になるようにスタートを置き換える。このようにすると、ゴールの見た目は同じになるので考えやすい。

[ABC436-E の解説](#)によれば、今回のパズルでは、解となるそれぞれの置換に必要な互換の回数はサイクル分解により計算でき、その答えはサイクルの数を C とすると 頂点数 $- C$ となる。解によってサイクルの個数 C は異なるかもしれないが、少なくとも、行うことが確定している操作をすると、必ず C が一つ増加する。

5.1.2. 行うことが確定している操作をすると、必ず C が一つ増加することの証明

ある頂点を置く場所が一か所に決まっているので、その頂点につけられる数字はどの解でも同じ数字になる。ここではその数を k と置く。

このとき、その行うことが確定している操作は、 k 番目にいない k を k 番目に入れ替える操作となる。

操作前は $k \rightarrow k$ 番目にいる要素 $\rightarrow \dots \rightarrow k$ の一つ前 $\rightarrow k$ というサイクルがあり、操作で移動するどちらの頂点も同じサイクルに属している。操作後は $k \rightarrow k$ のサイクルと、 k 番目にもともといた要素 $\rightarrow \dots \rightarrow k$ の一つ前だった要素 $\rightarrow k$ 番目にもともといた要素の二つに分裂する。従って C が一つ増加する。

そして、どの操作においても C は一つしか増加しないため、確定している操作に優劣はなく、どの操作を行っても結果は変わらない。

参考: [置換や互換に関する入門記事](#)

そしてさらに、正しくない位置にいる頂点を正しい位置に一つずつ移動していく形の最短の解が存在する。ということも証明できる。

5.1.3. 証明

C が一番大きくなるような順列を一つ選ぶ。この順列に従って、正しくない位置にいる頂点を正しい位置に一つずつ移動していくと、最短の解の一つが得られる。

操作は全て k 番目にいない k を k 番目に入れ替える操作となるので、上の説明と同様に C が一つ増加する。

操作前は $k \rightarrow k$ 番目にいる要素 $\rightarrow \dots \rightarrow k$ の一つ前 $\rightarrow k$ というサイクルがあり、操作で移動するどちらの頂点も同じサイクルに属している。操作後は $k \rightarrow k$ のサイクルと、 k 番目にもともといた要素 $\rightarrow \dots \rightarrow k$ の一つ前だった要素 $\rightarrow k$ 番目にもともといた要素の二つに分裂する。従って C が一つ増加する。